P.PORTO ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO	Exercícios Ficha Prática n.º3	Ano letivo 2024/2025	Data
	Curso Licenciatura em Engenharia Informática & Licenciatura em Segurança Informática em Redes de Computadores		
	Unidade Curricular Sistemas Operativos		

Observações
- *Thread* em Java

1- *Thread*

Uma *thread* pode ser vista como uma linha de execução de um processo. Um processo é algo que dispõe de um segmento de memória próprio e que executa de forma independente dos restantes processos. Um processo contém pelo menos uma linha de execução (ou *thread*). As *thread* são frequentemente consideradas como *lightweight processes* já que tanto os processos como as *thread* possibilitam a execução de código. Contudo, uma *thread* existe dentro do contexto de um processo e partilha o segmento de memória, ficheiros abertos e demais recursos, com as restantes *thread* (caso existam) do mesmo processo.

```
1  public class HelloRunnable implements Runnable {
2
3      public void run() {
4          System.out.println("Hello from a thread!");
5      }
6
7      public static void main(String args[]) {
8          (new Thread(new HelloRunnable())).start();
9      }
10
11 }
```

Listagem 1: Implementação da interface *Runnable*

Existem duas formas de se utilizarem *thread* em Java, sendo estas a implementação da *interface Runnable* (ver Listagem 1) ou a extensão da classe *Thread* (ver Listagem 2). O resultado é o mesmo, mas é mais comum utilizar-se a primeira opção já que a segunda limita a possibilidade da extensão da classe.

```
1  public class HelloThread extends Thread {
2
3      public void run() {
4          System.out.println("Hello from a thread!");
5      }
6
7      public static void main(String args[]) {
8          (new HelloThread()).start();
9      }
10
11 }
```

Listagem 2: Extensão da classe *Thread*

O modo de implementação é também similar e consiste na especificação do método *run()* em ambos os casos. O inicio da execução de uma *thread* também é igual e consiste na chamada ao método *start()*. A forma como se chama o método é ligeiramente diferente (ver linhas 8 das Listagens 1 e 2). Na prática, quando se invoca o método *start()* de uma *thread*, a JVM vai alocar memória e inicializar a *thread*, invocando de seguida o método *run()* da *thread*. Note-se que nunca se deve invocar o método *run()* diretamente, mas sim o método *start()*.

2- Exercícios

1. Elabore uma classe que crie um determinado número de *thread*. O número de *thread* deve ser passado por argumento da linha de comandos à classe principal. Cada *thread* deve limitar-se a imprimir o texto “Eu sou uma thread!".

2. Adapte a solução anterior para que cada *thread* tenha um nome diferente e que o utilize para identificar a linha que imprime. O texto que cada *thread* deve imprimir deve ser semelhante a:
   “\[Th0] Eu sou uma thread"
   “\[Th1] Eu sou uma thread"

3. Elabore uma classe que crie uma *thread*. A *thread* deve imprimir 10 linhas para o ecrã, imprimindo uma linha a cada segundo. A classe principal deve interromper a *thread* ao fim de um determinado número de segundos. O número de segundos deve ser passado por argumento.